

Competitive Programming

Section 70 Editorial: Bounded And Minimum-Cost Flows

Competitive Programming Course

Table of contents

1	How to use this editorial	3
2	A. Bounded Circulation	4
2.1	How to find it	4
2.2	Correctness	4
2.3	Complexity and tests	4
2.4	C++	4
2.5	Python	6
3	B. Exact-Cost Shipment	9
3.1	How to find it	9
3.2	Correctness	9
3.3	Complexity and tests	9
3.4	C++	9
3.5	Python	11
4	C. Quota Assignment	14
4.1	How to find it	14
4.2	Correctness	14
4.3	Complexity and tests	14
4.4	C++	14
4.5	Python	17
5	D. Minimum-Cost Bounded Circulation	20
5.1	Restatement and traps	20
5.2	Derivation	20
5.3	Pseudocode	20
5.4	Correctness proof	21
5.5	Complexity	21
5.6	Wrong answers and tests	21
5.7	C++	21
5.8	Python	24
6	Stress-testing summary	26

1 How to use this editorial

For each problem, identify the reduction before reading the code. The proof paragraph names the invariant that should have appeared in your own derivation.

2 A. Bounded Circulation

Subtract every lower bound and accumulate vertex balance. The super-source flow is feasible exactly when all positive balances can be routed to all deficits through the remaining capacities. Add the lower bounds back conceptually. Complexity is one maximum-flow computation.

2.1 How to find it

The obstacle is that zero flow is no longer a legal starting point. Separate what is forced from what remains optional: write every edge flow as its lower bound plus an extra amount. The forced parts no longer conserve flow locally, so the only missing information is one signed imbalance per vertex.

2.2 Correctness

Given a valid bounded circulation, subtracting every lower bound leaves a flow within capacities **high-low**. Its imbalance at each vertex is exactly repaired by the auxiliary super-source and super-sink edges, so all super-source edges can be saturated. Conversely, a saturating auxiliary flow repairs every imbalance. Removing the auxiliary edges and adding each lower bound back gives conservation and respects both bounds. Thus the reduction is equivalent in both directions.

2.3 Complexity and tests

The transformed graph has $n+2$ vertices and $m+O(n)$ edges; the cost is one Dinic maximum-flow run. Test isolated forced edges, **low=high**, parallel edges, zero bounds, and total balance zero with locally impossible routing.

2.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

struct Dinic {
    struct Edge {
        int to, rev;
        int64 cap;
    };
};
```

```

vector<vector<Edge>> g;
vector<int> level, it;
Dinic(int n) : g(n), level(n), it(n) {}
void add_edge(int u, int v, int64 cap) {
    Edge a{v, (int)g[v].size(), cap};
    Edge b{u, (int)g[u].size(), 0};
    g[u].push_back(a);
    g[v].push_back(b);
}
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    queue<int> q;
    level[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto &e : g[u])
            if (e.cap && level[e.to] < 0)
                level[e.to] = level[u] + 1, q.push(e.to);
    }
    return level[t] >= 0;
}
int64 dfs(int u, int t, int64 pushed) {
    if (u == t)
        return pushed;
    for (int &i = it[u]; i < (int)g[u].size(); ++i) {
        Edge &e = g[u][i];
        if (!e.cap || level[e.to] != level[u] + 1)
            continue;
        int64 take = dfs(e.to, t, min(pushed, e.cap));
        if (take) {
            e.cap -= take;
            g[e.to][e.rev].cap += take;
            return take;
        }
    }
    return 0;
}
int64 flow(int s, int t) {
    int64 ans = 0, pushed;
    while (bfs(s, t)) {
        fill(it.begin(), it.end(), 0);
        while ((pushed = dfs(s, t, (1LL << 62))))
            ans += pushed;
    }
    return ans;
}
};

int main() {

```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);
int n, m;
if (!(cin >> n >> m))
    return 0;
int ss = n, tt = n + 1;
Dinic dinic(n + 2);
vector<int64> balance(n);
for (int i = 0, u, v; i < m; ++i) {
    int64 low, high;
    cin >> u >> v >> low >> high;
    --u;
    --v;
    dinic.add_edge(u, v, high - low);
    balance[u] -= low;
    balance[v] += low;
}
int64 need = 0;
for (int v = 0; v < n; ++v) {
    if (balance[v] > 0)
        dinic.add_edge(ss, v, balance[v]), need += balance[v];
    if (balance[v] < 0)
        dinic.add_edge(v, tt, -balance[v]);
}
cout << (dinic.flow(ss, tt) == need ? "YES\n" : "NO\n");
}

```

2.5 Python

```

import collections
import sys

class Dinic:
    def __init__(self, n):
        self.g = [[] for _ in range(n)]

    def add_edge(self, u, v, cap):
        self.g[u].append([v, cap, len(self.g[v])])
        self.g[v].append([u, 0, len(self.g[u]) - 1])

    def flow(self, s, t):
        answer = 0
        while True:
            level = [-1] * len(self.g)
            level[s] = 0
            q = collections.deque([s])
            while q:

```

```

        u = q.popleft()
        for v, cap, _ in self.g[u]:
            if cap and level[v] < 0:
                level[v] = level[u] + 1
                q.append(v)
    if level[t] < 0:
        return answer
    it = [0] * len(self.g)

    def dfs(u, pushed):
        if u == t:
            return pushed
        while it[u] < len(self.g[u]):
            e = self.g[u][it[u]]
            v, cap, rev = e
            if cap and level[v] == level[u] + 1:
                take = dfs(v, min(pushed, cap))
                if take:
                    e[1] -= take
                    self.g[v][rev][1] += take
                    return take
            it[u] += 1
        return 0

    while pushed := dfs(s, 10**30):
        answer += pushed

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m = data[:2]
    dinic = Dinic(n + 2)
    balance = [0] * n
    at = 2
    for _ in range(m):
        u, v, low, high = data[at : at + 4]
        at += 4
        u -= 1
        v -= 1
        dinic.add_edge(u, v, high - low)
        balance[u] -= low
        balance[v] += low
    need = 0
    for v, value in enumerate(balance):
        if value > 0:
            dinic.add_edge(n, v, value)
            need += value
        elif value < 0:
            dinic.add_edge(v, n + 1, -value)

```

```
    print("YES" if dinic.flow(n, n + 1) == need else "NO")

if __name__ == "__main__":
    main()
```

3 B. Exact-Cost Shipment

A residual reverse edge permits replacing earlier choices. Initial Bellman–Ford potentials make every reachable reduced cost nonnegative; Dijkstra then finds each cheapest augmentation. Updating reached potentials by their Dijkstra distances preserves feasibility. Stop after exactly F units or report impossibility.

3.1 How to find it

A greedy cheapest original path fails because a later unit may need to undo an earlier route. That observation asks for a residual graph: every used edge creates a reverse edge of opposite cost. We still want shortest augmenting paths, so potentials are introduced solely to make Dijkstra legal.

3.2 Correctness

Feasible potentials make all reduced residual costs nonnegative, and path reweighting changes every s – t path cost by the same constant. Dijkstra therefore chooses a truly cheapest residual augmenting path. Updating reached potentials with shortest reduced distances preserves non-negative reduced costs by the triangle inequality. The standard residual optimality criterion then shows inductively that after each augmentation the current flow has minimum cost among flows of its value. Hence the state after F units is the required optimum.

3.3 Complexity and tests

With integral capacities, at most F augmentations are needed here, for $O(F E \log V)$ after the initial shortest-distance computation. Test negative input edges, a case where a reverse residual edge changes the first choice, parallel routes, zero requested flow, and insufficient capacity.

3.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
const int64 INF = (1LL << 62);

struct MCF {
    struct Edge {
        int to, rev, cap, initial;
```

```

    int64 cost;
};
vector<vector<Edge>> g;
MCF(int n) : g(n) {}
pair<int, int> add_edge(int u, int v, int cap, int64 cost) {
    int id = g[u].size();
    g[u].push_back({v, (int)g[v].size(), cap, cap, cost});
    g[v].push_back({u, id, 0, 0, -cost});
    return {u, id};
}
pair<int, int64> flow(int s, int t, int limit) {
    int n = g.size(), sent = 0;
    int64 cost = 0;
    vector<int64> pot(n, INF), dist(n);
    pot[s] = 0;
    for (int rep = 0; rep < n; ++rep) {
        bool changed = false;
        for (int u = 0; u < n; ++u)
            if (pot[u] < INF)
                for (auto &e : g[u])
                    if (e.cap && pot[e.to] > pot[u] + e.cost)
                        pot[e.to] = pot[u] + e.cost, changed = true;
        if (!changed)
            break;
    }
    for (auto &x : pot)
        if (x == INF)
            x = 0;
    vector<int> pv(n), pe(n);
    while (sent < limit) {
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        priority_queue<pair<int64, int>, vector<pair<int64, int>>,
            greater<pair<int64, int>>>
            pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d != dist[u])
                continue;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto &e = g[u][i];
                int64 nd = d + e.cost + pot[u] - pot[e.to];
                if (e.cap && nd < dist[e.to])
                    dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i, pq.push({nd, e.to});
            }
        }
        if (dist[t] == INF)
            break;
        for (int v = 0; v < n; ++v)

```

```

        if (dist[v] < INF)
            pot[v] += dist[v];
        int add = limit - sent;
        for (int v = t; v != s; v = pv[v])
            add = min(add, g[pv[v]][pe[v]].cap);
        for (int v = t; v != s; v = pv[v]) {
            Edge &e = g[pv[v]][pe[v]];
            cost += (int64)add * e.cost;
            e.cap -= add;
            g[v][e.rev].cap += add;
        }
        sent += add;
    }
    return {sent, cost};
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, need;
    if (!(cin >> n >> m >> need))
        return 0;
    int s, t;
    cin >> s >> t;
    --s;
    --t;
    MCF mcf(n);
    for (int i = 0, u, v, cap; i < m; ++i) {
        int64 cost;
        cin >> u >> v >> cap >> cost;
        mcf.add_edge(--u, --v, cap, cost);
    }
    auto [sent, cost] = mcf.flow(s, t, need);
    if (sent < need)
        cout << "IMPOSSIBLE\n";
    else
        cout << cost << '\n';
}

```

3.5 Python

```

import heapq
import sys

class MCF:
    def __init__(self, n):

```

```

self.g = [[] for _ in range(n)]

def add_edge(self, u, v, cap, cost):
    ref = (u, len(self.g[u]))
    self.g[u].append([v, cap, cost, len(self.g[v]), cap])
    self.g[v].append([u, 0, -cost, len(self.g[u]) - 1, 0])
    return ref

def flow(self, s, t, limit):
    n = len(self.g)
    inf = 10**40
    potential = [inf] * n
    potential[s] = 0
    for _ in range(n):
        changed = False
        for u in range(n):
            if potential[u] == inf:
                continue
            for v, cap, cost, _, _ in self.g[u]:
                if cap and potential[v] > potential[u] + cost:
                    potential[v] = potential[u] + cost
                    changed = True
        if not changed:
            break
    potential = [0 if x == inf else x for x in potential]
    sent = total = 0
    while sent < limit:
        dist = [inf] * n
        prev = [None] * n
        dist[s] = 0
        pq = [(0, s)]
        while pq:
            d, u = heapq.heappop(pq)
            if d != dist[u]:
                continue
            for i, (v, cap, cost, _, _) in enumerate(self.g[u]):
                nd = d + cost + potential[u] - potential[v]
                if cap and nd < dist[v]:
                    dist[v] = nd
                    prev[v] = (u, i)
                    heapq.heappush(pq, (nd, v))
        if dist[t] == inf:
            break
        for v in range(n):
            if dist[v] < inf:
                potential[v] += dist[v]
        add = limit - sent
        v = t
        while v != s:
            u, i = prev[v]
            add = min(add, self.g[u][i][1])

```

```

        v = u
    v = t
    while v != s:
        u, i = prev[v]
        e = self.g[u][i]
        total += add * e[2]
        e[1] -= add
        self.g[v][e[3]][1] += add
        v = u
    sent += add
    return sent, total

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m, need, s, t = data[:5]
    mcf = MCF(n)
    at = 5
    for _ in range(m):
        u, v, cap, cost = data[at : at + 4]
        at += 4
        mcf.add_edge(u - 1, v - 1, cap, cost)
    sent, cost = mcf.flow(s - 1, t - 1, need)
    print(cost if sent == need else "IMPOSSIBLE")

if __name__ == "__main__":
    main()

```

4 C. Quota Assignment

Send one unit through each worker. Project-to-sink capacity enforces upper quotas. Give the first `low[j]` parallel slots a reward `-BIG`, where `BIG` exceeds every possible difference in real profit, and verify all rewarded slots are saturated. The remaining min-cost objective is precisely negative total profit.

4.1 How to find it

“Every worker exactly once” naturally means one unit leaving every worker; eligibility edges encode the choices and their costs are negative profits. Upper quotas are capacities. To use a plain min-cost-flow implementation for lower quotas, turn the first `low[j]` project slots into lexicographically mandatory slots with reward `-BIG`.

4.2 Correctness

Every integral flow of value `n` decomposes into one eligible project choice per worker, and every assignment yields such a flow. Project capacities enforce the upper bounds. If a quota-feasible assignment exists, it fills all mandatory slots. Because `BIG` exceeds the maximum possible difference between two total profits, every minimum-cost flow first maximizes the number of mandatory slots filled and only then maximizes real profit. Verifying their saturation rejects exactly the infeasible instances; removing their constant reward recovers the true optimum.

4.3 Complexity and tests

There are $O(n+p)$ vertices and $O(n+m+p)$ edges. Successive shortest paths take $O(n \cdot E \cdot \log V)$. Test a worker with no choices, tight quotas, infeasible lower quotas despite sufficient total capacity, and an optimum using a lower-profit edge to satisfy a project minimum.

4.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
const int64 INF = (1LL << 62);

struct MCF {
    struct Edge {
        int to, rev, cap, initial;
```

```

    int64 cost;
};
vector<vector<Edge>> g;
MCF(int n) : g(n) {}
pair<int, int> add_edge(int u, int v, int cap, int64 cost) {
    int id = g[u].size();
    g[u].push_back({v, (int)g[v].size(), cap, cap, cost});
    g[v].push_back({u, id, 0, 0, -cost});
    return {u, id};
}
pair<int, int64> flow(int s, int t, int limit) {
    int n = g.size(), sent = 0;
    int64 cost = 0;
    vector<int64> pot(n, INF), dist(n);
    pot[s] = 0;
    for (int rep = 0; rep < n; ++rep) {
        bool changed = false;
        for (int u = 0; u < n; ++u)
            if (pot[u] < INF)
                for (auto &e : g[u])
                    if (e.cap && pot[e.to] > pot[u] + e.cost)
                        pot[e.to] = pot[u] + e.cost, changed = true;
        if (!changed)
            break;
    }
    for (auto &x : pot)
        if (x == INF)
            x = 0;
    vector<int> pv(n), pe(n);
    while (sent < limit) {
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        priority_queue<pair<int64, int>, vector<pair<int64, int>>,
            greater<pair<int64, int>>>
            pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d != dist[u])
                continue;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto &e = g[u][i];
                int64 nd = d + e.cost + pot[u] - pot[e.to];
                if (e.cap && nd < dist[e.to])
                    dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i, pq.push({nd, e.to});
            }
        }
        if (dist[t] == INF)
            break;
        for (int v = 0; v < n; ++v)

```

```

        if (dist[v] < INF)
            pot[v] += dist[v];
        int add = limit - sent;
        for (int v = t; v != s; v = pv[v])
            add = min(add, g[pv[v]][pe[v]].cap);
        for (int v = t; v != s; v = pv[v]) {
            Edge &e = g[pv[v]][pe[v]];
            cost += (int64)add * e.cost;
            e.cap -= add;
            g[v][e.rev].cap += add;
        }
        sent += add;
    }
    return {sent, cost};
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, p, m;
    if (!(cin >> n >> p >> m))
        return 0;
    vector<int> low(p), high(p);
    int required = 0;
    for (int j = 0; j < p; ++j)
        cin >> low[j] >> high[j], required += low[j];
    struct Choice {
        int w, j, profit;
    };
    vector<Choice> choices(m);
    int max_profit = 0;
    for (auto &[w, j, profit] : choices) {
        cin >> w >> j >> profit;
        --w;
        --j;
        max_profit = max(max_profit, profit);
    }
    int s = n + p, t = s + 1;
    MCF mcf(t + 1);
    for (int w = 0; w < n; ++w)
        mcf.add_edge(s, w, 1, 0);
    for (auto [w, j, profit] : choices)
        mcf.add_edge(w, n + j, 1, -profit);
    int64 big = (int64)(n + 1) * (max_profit + 1);
    vector<pair<int, int>> mandatory;
    for (int j = 0; j < p; ++j) {
        mandatory.push_back(mcf.add_edge(n + j, t, low[j], -big));
        mcf.add_edge(n + j, t, high[j] - low[j], 0);
    }
    auto [sent, cost] = mcf.flow(s, t, n);

```

```

bool ok = sent == n;
for (auto [u, id] : mandatory)
    ok &= mcf.g[u][id].cap == 0;
if (!ok)
    cout << "IMPOSSIBLE\n";
else
    cout << -cost - big * required << '\n';
}

```

4.5 Python

```

import heapq
import sys

class MCF:
    def __init__(self, n):
        self.g = [[] for _ in range(n)]

    def add_edge(self, u, v, cap, cost):
        ref = (u, len(self.g[u]))
        self.g[u].append([v, cap, cost, len(self.g[v]), cap])
        self.g[v].append([u, 0, -cost, len(self.g[u]) - 1, 0])
        return ref

    def flow(self, s, t, limit):
        n = len(self.g)
        inf = 10**40
        potential = [inf] * n
        potential[s] = 0
        for _ in range(n):
            changed = False
            for u in range(n):
                if potential[u] == inf:
                    continue
                for v, cap, cost, _, _ in self.g[u]:
                    if cap and potential[v] > potential[u] + cost:
                        potential[v] = potential[u] + cost
                        changed = True
            if not changed:
                break
        potential = [0 if x == inf else x for x in potential]
        sent = total = 0
        while sent < limit:
            dist = [inf] * n
            prev = [None] * n
            dist[s] = 0
            pq = [(0, s)]

```

```

        while pq:
            d, u = heapq.heappop(pq)
            if d != dist[u]:
                continue
            for i, (v, cap, cost, _, _) in enumerate(self.g[u]):
                nd = d + cost + potential[u] - potential[v]
                if cap and nd < dist[v]:
                    dist[v] = nd
                    prev[v] = (u, i)
                    heapq.heappush(pq, (nd, v))
        if dist[t] == inf:
            break
        for v in range(n):
            if dist[v] < inf:
                potential[v] += dist[v]
        add = limit - sent
        v = t
        while v != s:
            u, i = prev[v]
            add = min(add, self.g[u][i][1])
            v = u
        v = t
        while v != s:
            u, i = prev[v]
            e = self.g[u][i]
            total += add * e[2]
            e[1] -= add
            self.g[v][e[3]][1] += add
            v = u
        sent += add
    return sent, total

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, p, m = data[:3]
    at = 3
    low = []
    high = []
    for _ in range(p):
        lo, hi = data[at : at + 2]
        at += 2
        low.append(lo)
        high.append(hi)
    choices = []
    max_profit = 0
    for _ in range(m):
        w, j, profit = data[at : at + 3]
        at += 3

```

```

        choices.append((w - 1, j - 1, profit))
    max_profit = max(max_profit, profit)
s = n + p
t = s + 1
mcf = MCF(t + 1)
for w in range(n):
    mcf.add_edge(s, w, 1, 0)
for w, j, profit in choices:
    mcf.add_edge(w, n + j, 1, -profit)
big = (n + 1) * (max_profit + 1)
mandatory = []
for j in range(p):
    mandatory.append(mcf.add_edge(n + j, t, low[j], -big))
    mcf.add_edge(n + j, t, high[j] - low[j], 0)
sent, cost = mcf.flow(s, t, n)
ok = sent == n and all(mcf.g[u][i][1] == 0 for u, i in mandatory)
print(-cost - big * sum(low) if ok else "IMPOSSIBLE")

if __name__ == "__main__":
    main()

```

5 D. Minimum-Cost Bounded Circulation

5.1 Restatement and traps

Every edge has both a mandatory lower amount and an optional remaining capacity. We need conservation and minimum cost simultaneously. Merely finding any feasible bounded circulation and then running an unrelated optimization can lock in a nonoptimal repair. The nonnegative-cost guarantee is important: once all balance requirements are satisfied, adding an extra circulation cannot lower the objective.

5.2 Derivation

Send every lower bound first. Add its cost to `mandatory_cost`, replace the edge capacity by `high-low`, and update the endpoints' balances. Add a super-source to every vertex with excess mandatory inflow and connect every deficit vertex to the super-sink.

Unlike feasibility problem A, route the required auxiliary flow with minimum-cost flow. The answer is

```
mandatory lower-bound cost + minimum repair cost.
```

If the auxiliary flow cannot saturate the total positive balance, no bounded circulation exists.

5.3 Pseudocode

```
base_cost = 0
for edge (u,v,low,high,cost):
    base_cost += low * cost
    balance[u] -= low
    balance[v] += low
    add residual edge u -> v with capacity high-low and given cost

connect super-source to positive balances
connect negative balances to super-sink
send exactly sum(positive balances) units at minimum cost
if not enough flow: IMPOSSIBLE
else: base_cost + repair_cost
```

5.4 Correctness proof

Lemma 1. Feasible auxiliary flows correspond exactly to feasible bounded circulations after lower bounds are restored.

Proof. This is the balance equivalence from problem A. Subtracting lower bounds from a bounded circulation produces a residual flow repairing exactly the recorded imbalances. Conversely, a saturating auxiliary flow repairs all imbalances; removing the auxiliary edges and restoring lower bounds gives conservation and respects every interval. QED.

Lemma 2. Among all feasible bounded circulations, the algorithm finds one with minimum cost.

Proof. Every solution pays the same mandatory lower-bound cost. By Lemma 1, its remaining edge amounts correspond to an auxiliary flow saturating the required balance, with identical additional cost. Successive shortest paths find the cheapest such auxiliary flow. Since all original costs are nonnegative, an additional circulation after satisfying balances cannot improve the cost. Thus minimizing repair cost minimizes total cost. QED.

The two lemmas establish the reported optimum or impossibility.

5.5 Complexity

The transformed network has $n+2$ vertices and $m+O(n)$ edges. With required auxiliary amount F , successive shortest paths take $O(F \cdot E \cdot \log V)$ here.

5.6 Wrong answers and tests

- Forgetting to add the fixed lower-bound cost.
- Reversing which balances connect to the super-source.
- Optimizing before enforcing saturation of every balance edge.
- Allowing negative input costs without also cancelling negative cycles after the required flow; that is a more general problem than this statement.

Test $\text{low}=\text{high}$, an impossible single forced edge, parallel repair routes with different costs, a zero-balance instance, and a case where a longer route is cheaper.

5.7 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
const int64 INF = (1LL << 62);

struct MinCostFlow {
    struct Edge {
        int to, reverse, capacity;
        int64 cost;
```

```

};
vector<vector<Edge>> graph;

explicit MinCostFlow(int n) : graph(n) {}

void add_edge(int u, int v, int capacity, int64 cost) {
    int ui = graph[u].size();
    int vi = graph[v].size();
    graph[u].push_back({v, vi, capacity, cost});
    graph[v].push_back({u, ui, 0, -cost});
}

pair<int, int64> flow(int source, int sink, int required) {
    int n = graph.size();
    int sent = 0;
    int64 total_cost = 0;
    vector<int64> potential(n), distance(n);
    vector<int> previous_vertex(n), previous_edge(n);

    while (sent < required) {
        fill(distance.begin(), distance.end(), INF);
        distance[source] = 0;
        priority_queue<pair<int64, int>, vector<pair<int64, int>>,
            greater<pair<int64, int>>> queue;
        queue.push({0, source});

        while (!queue.empty()) {
            auto [current_distance, u] = queue.top();
            queue.pop();
            if (current_distance != distance[u]) continue;
            for (int index = 0; index < (int)graph[u].size(); ++index) {
                const Edge& edge = graph[u][index];
                if (!edge.capacity) continue;
                int64 candidate = current_distance + edge.cost
                    + potential[u] - potential[edge.to];
                if (candidate < distance[edge.to]) {
                    distance[edge.to] = candidate;
                    previous_vertex[edge.to] = u;
                    previous_edge[edge.to] = index;
                    queue.push({candidate, edge.to});
                }
            }
        }
        if (distance[sink] == INF) break;
        for (int v = 0; v < n; ++v) {
            if (distance[v] < INF) potential[v] += distance[v];
        }

        int add = required - sent;
        for (int v = sink; v != source; v = previous_vertex[v]) {
            add = min(add, graph[previous_vertex[v]][previous_edge[v]].capacity);
        }
    }
}

```

```

        }
        for (int v = sink; v != source; v = previous_vertex[v]) {
            Edge& edge = graph[previous_vertex[v]][previous_edge[v]];
            total_cost += int64(add) * edge.cost;
            edge.capacity -= add;
            graph[v][edge.reverse].capacity += add;
        }
        sent += add;
    }
    return {sent, total_cost};
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    int source = n;
    int sink = n + 1;
    MinCostFlow network(n + 2);
    vector<int> balance(n);
    int64 mandatory_cost = 0;

    for (int edge = 0; edge < m; ++edge) {
        int u, v, low, high, cost;
        cin >> u >> v >> low >> high >> cost;
        --u;
        --v;
        balance[u] -= low;
        balance[v] += low;
        mandatory_cost += int64(low) * cost;
        network.add_edge(u, v, high - low, cost);
    }

    int required = 0;
    for (int v = 0; v < n; ++v) {
        if (balance[v] > 0) {
            network.add_edge(source, v, balance[v], 0);
            required += balance[v];
        } else if (balance[v] < 0) {
            network.add_edge(v, sink, -balance[v], 0);
        }
    }

    auto [sent, extra_cost] = network.flow(source, sink, required);
    if (sent != required) cout << "IMPOSSIBLE\n";
    else cout << mandatory_cost + extra_cost << '\n';
}

```

5.8 Python

```
import heapq
import sys

class MinCostFlow:
    def __init__(self, n: int):
        self.graph = [[] for _ in range(n)]

    def add_edge(self, u: int, v: int, capacity: int, cost: int) -> None:
        self.graph[u].append([v, capacity, cost, len(self.graph[v])])
        self.graph[v].append([u, 0, -cost, len(self.graph[u]) - 1])

    def flow(self, source: int, sink: int, required: int):
        n = len(self.graph)
        potential = [0] * n
        sent = total_cost = 0
        infinity = 10**30

        while sent < required:
            distance = [infinity] * n
            previous = [None] * n
            distance[source] = 0
            queue = [(0, source)]
            while queue:
                current_distance, u = heapq.heappop(queue)
                if current_distance != distance[u]:
                    continue
                for index, edge in enumerate(self.graph[u]):
                    v, capacity, cost, _ = edge
                    if not capacity:
                        continue
                    candidate = current_distance + cost + potential[u] - potential[v]
                    if candidate < distance[v]:
                        distance[v] = candidate
                        previous[v] = (u, index)
                        heapq.heappush(queue, (candidate, v))
            if distance[sink] == infinity:
                break
            for v in range(n):
                if distance[v] < infinity:
                    potential[v] += distance[v]

            add = required - sent
            v = sink
            while v != source:
                u, index = previous[v]
                add = min(add, self.graph[u][index][1])
                v = u
```

```

        v = sink
        while v != source:
            u, index = previous[v]
            edge = self.graph[u][index]
            total_cost += add * edge[2]
            edge[1] -= add
            self.graph[v][edge[3]][1] += add
            v = u
        sent += add
    return sent, total_cost

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m = data[:2]
    source, sink = n, n + 1
    network = MinCostFlow(n + 2)
    balance = [0] * n
    mandatory_cost = 0
    at = 2
    for _ in range(m):
        u, v, low, high, cost = data[at : at + 5]
        at += 5
        u -= 1
        v -= 1
        balance[u] -= low
        balance[v] += low
        mandatory_cost += low * cost
        network.add_edge(u, v, high - low, cost)

    required = 0
    for v, amount in enumerate(balance):
        if amount > 0:
            network.add_edge(source, v, amount, 0)
            required += amount
        elif amount < 0:
            network.add_edge(v, sink, -amount, 0)

    sent, extra_cost = network.flow(source, sink, required)
    print(mandatory_cost + extra_cost if sent == required else "IMPOSSIBLE")

if __name__ == "__main__":
    main()

```

6 Stress-testing summary

For D , the tiny oracle enumerates every flow value in every edge interval, checks conservation literally, and takes the cheapest valid assignment. It shares neither the balance transformation nor the residual algorithm with the reference solution.